

DAOA Experiment List for Practical Exam 2025-26

1. Implement Merge sort and analyze its time complexity.
2. Implement Quick sort and analyze its time complexity.
3. Implement randomized Quick Sort.
4. Implementation of fractional knapsack problem by applying a greedy approach.
5. Implement Dijkstra's algorithm to find the single source shortest path.
6. Implement Longest Common Subsequence algorithm.
7. Implement Floyd warshall algorithm to find all pair shortest path.
8. Implement N-queen problem to demonstrate Backtracking approach.
9. Implement and demonstrate the working of the branch and bound algorithm with the implementation of 0/1 Knapsack.
10. Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.
11. We define a magic square to be an $n \times n$ matrix of distinct positive integers from 1 to n^2 where the sum of any row, column, or diagonal of length n is always equal to the same number: the magic constant. You will be given a 3×3 matrix S of integers in the inclusive range $[1, 9]$. We can convert any digit a to any other digit b in the range $[1, 9]$ at cost of $|a - b|$. Given S , convert it into a magic square at minimal cost. Print this cost on a new line.
12. There is an undirected tree where each vertex is numbered from 1 to n , and each contains a data value. The sum of a tree is the sum of all its nodes' data values. If an edge is cut, two smaller trees are formed. The difference between two trees is the absolute value of the difference in their sums. Given a tree, determine which edge to cut so that the resulting trees have a minimal difference between them, then return that difference.
13. Implementation of Sum of Subset problem to demonstrate Backtracking approach.
14. Implementation of Job Sequencing with deadline using greedy approach.
15. Implementation of Bellman Ford algorithm using Dynamic programming
16. Implementation of Graph Coloring problem to demonstrate Backtracking approach.
17. Implementation of Kruskal's algorithm using greedy approach.
18. Implementation of Prim's algorithm using greedy approach.
19. Implementation of an algorithm to solve Activity selection problem using greedy approach.
20. Constructing B tree with the insertion of elements to satisfy its properties.

21. Implement 0/1 knapsack problem using dynamic programming.
22. Implement a greedy algorithm to schedule delivery jobs for a courier company to maximize total profit before deadlines.
23. A delivery company uses a drone to distribute parcels to several drop-off locations. The drone must start from its base, visit every drop-off point exactly once, and return to the base while minimizing the total travel distance. Implement a Dynamic Programming-based Travelling Salesperson (TSP) algorithm to compute the optimal delivery route for the drone.
24. A logistics company needs to pack items into a shipment container that has a fixed weight capacity. You are given a list of item weights, and the goal is to identify all possible combinations of items whose total weight exactly matches the container's capacity. Implement a backtracking algorithm to generate every valid subset that sums to the target weight.
25. A digital mapping system needs to display different countries using colors. To avoid confusion, no two countries that share a border should be displayed in the same color. Design and implement a graph coloring algorithm to achieve this and find the minimum number of colors required.
26. A traveler is carrying a backpack that can hold only a limited amount of weight. A list of items is provided, each with a specific weight and value. Use the 0/1 Knapsack dynamic programming approach to determine which items should be chosen to achieve the highest total value without exceeding the backpack's weight capacity.
27. A company has a salesperson who must visit several client locations starting and ending at the office. The goal is to determine the route that covers all client visits with the least possible travel distance. Implement a Dynamic Programming-based Travelling Salesperson (TSP) algorithm to compute the shortest possible tour for the salesperson.
28. A tourist plans to visit several locations in a specific order, and each stage of the journey offers multiple possible routes with different travel costs. Using the Multistage Graph Dynamic Programming approach, determine the minimum-cost path the tourist should follow to complete the trip in the required sequence.
29. A housing society requires an electricity distribution network connecting all houses with the minimum total wiring cost. Implement Prim's algorithm to design a minimum spanning tree that determines the optimal connections between houses to distribute electricity efficiently.
30. A university campus plans to set up an internet backbone connecting all its buildings. Each possible connection between buildings has a specific installation cost. Apply Prim's algorithm to design a minimum-cost network that connects all campus buildings efficiently.
31. A smart farming system requires connecting multiple sensors across the farm with wiring or wireless links. Each possible connection has an associated cost. Implement Kruskal's algorithm to determine the cheapest layout that connects all sensors efficiently.

32. A conference organizer has multiple talks to schedule in a single hall, each with a start and end time. Implement a greedy-based activity selection algorithm to select the maximum number of talks that can be held in the hall without any time conflicts.
33. A digital payment application needs to calculate the minimum number of coins required to make a specific payment amount. The system has an unlimited supply of given coin denominations. Design and implement a Dynamic Programming solution to determine the minimum coins needed.
34. A vending machine must return change efficiently to customers. Given different coin values and a target amount, implement a Dynamic Programming approach that computes the minimum number of coins required to make the exact change.
35. A cashier system needs to generate all possible ways to make a specific payment amount using a set of available coins, where each coin is allowed to be used only one time. Write a Dynamic Programming algorithm to solve this problem.
36. Create a B-tree by inserting elements while ensuring that all B-tree properties are maintained.
37. Construct a B-tree through successive insertions of keys such that the structure always satisfies the rules of a B-tree.
38. A budgeting application needs to verify whether a combination of expense amounts can exactly match a given budget value. Write a program to solve this problem using the Sum of Subsets algorithm with Backtracking.
39. A bioinformatics system compares two DNA sequences to identify common genetic patterns. Design and implement a program using the Longest Common Subsequence (LCS) algorithm to determine the longest matching subsequence between two given DNA sequences.
40. A delivery company has a truck with limited carrying capacity. Each package has a weight and a profit value, and the company is allowed to take fractions of packages if needed. Write a program using the Greedy approach to maximize the total profit.
41. Design and code a solution for the N-Queens problem using Backtracking to show how conflicts can be avoided.
42. A software company receives several client tasks, each with a fixed deadline and payment amount. The company can complete only one task per time slot. Design and implement a program using the Greedy approach to determine the most profitable job sequence.
43. A data processing center must execute a set of tasks where each task has a deadline and an associated profit. Only one task can be done at a time. Implement the Job Sequencing with Deadlines algorithm using a Greedy strategy to get the maximum profit.

44. A navigation system needs to find the shortest path between cities in a road network where some road segments may have negative travel costs. Design and implement the Bellman–Ford algorithm using Dynamic Programming to solve this problem.